SkillGuardGraph: Cross-Layer Evidence Fusion for Detecting and Containing Malicious Skills in LLM Agent Ecosystems

Anonymous Authors

Abstract

Large language model (LLM) agents increasingly rely on external skills—tools, connectors, actions, MCP servers, and graph nodes—to access private data, invoke APIs, modify files, and maintain persistent state. This capability expansion creates a new security problem: malicious skills can exploit gaps between declared metadata, implemented behavior, granted permissions, runtime outputs, user approval, and post-approval updates. Existing defenses typically operate at a single layer: metadata scanning, prompt-injection filtering, static analysis, sandbox probing, or human-in-the-loop approval. We argue that malicious skills are compositional objects whose most damaging behaviors emerge across lifecycle boundaries.

We present **SkillGuardGraph**, a cross-layer evidence fusion framework for detecting and containing malicious skills in LLM agent ecosystems. SkillGuardGraph represents skill metadata, implementation summaries, sandbox observations, runtime provenance, permission scopes, approval dialogs, and version histories as a typed evidence graph, then enforces source-aware, permission-aware, and version-aware constraints over tool-call chains. We introduce a compositional benchmark spanning seven attack classes with 4,010 samples, and evaluate against metadata-only, static-only, sandbox-only, runtime-only, naive union, weighted voting, and LLM-judge baselines. In this synthetic benchmark, SkillGuardGraph reaches 100.0% precision and recall with 0.0% FPR, compared with naive union's 100.0% FPR and weighted voting's 20.8% FPR. Runtime enforcement blocks all malicious traces in the current trace-replay suite while preserving 100.0% benign task success. We further report a passive real-world measurement over 1,000 public MCP-related repositories with data-carded provenance, manual triage of all HIGH-severity findings, and no confirmed vulnerabilities disclosed from passive evidence alone.

1 Introduction

LLM agents are no longer isolated chat systems. They increasingly operate through external skills: OpenAI Actions and Apps, Anthropic connectors and MCP servers, Microsoft Copilot connectors and plugins, LangChain tools and graphs, AutoGPT blocks and agents, and Google Vertex AI Agent Builder extensions. These skills act as discoverable and reusable capability units outside the model's core reasoning loop. They connect models to files, repositories, enterprise knowledge bases, cloud APIs, email systems, ticketing systems, and local execution environments.

This shift changes the threat model. Prompt injection is no longer only about causing a model to produce unsafe text. A malicious or compromised skill can manipulate tool selection, request excessive permissions, return untrusted instructions as context, alter persistent memory or configuration, and exploit differences between what the user approves and what the execution layer actually performs.

A common defensive response is to stack several controls: scan the manifest, scan the code, run a sandbox, require approval, and log the event. This is necessary but insufficient. The critical security property often does not belong to one tool or one layer. It belongs to a sequence:

```
untrusted web page -> model context -> internal search tool
  -> confidential result -> third-party summarizer -> external send
```

Every individual step can appear legitimate, while the composed flow violates a policy. Similarly, a skill may be approved as read-only, then later update its handler, expand OAuth scopes, or return instruction-like output that affects high-privilege actions. These behaviors cannot be reliably captured by a single metadata classifier or a static analyzer.

Thesis. Malicious skills exploit cross-layer trust gaps that are invisible to single-layer defenses. Effective defense requires evidence fusion across declaration, implementation, permission, runtime provenance, approval integrity, persistence, and version drift.

1.1 Contributions

1. **Threat taxonomy.** We define a compositional taxonomy of seven malicious skill behaviors: capability laundering, cross-skill confused deputy, delayed rug-pull, consent laundering, persistence pivot, split exfiltration, and scope inflation (§4).
2. **Evidence graph and constraint system.** We introduce a typed cross-layer evidence graph with source-aware and permission-aware constraints over tool-call chains (§5, §6).
3. **Benchmark.** We propose a lifecycle-complete benchmark with 4,010 samples covering seven attack classes, benign-paired variants, and trace-grounded validators (§7).
4. **Evaluation.** We provide a comprehensive evaluation comparing graph fusion with seven baselines, an ablation study, runtime defense metrics, paired significance tests, failure analysis, and ecosystem measurement of 1,200 synthetic samples plus 1,000 public MCP-related GitHub repositories (§8, §9).

2 Background

2.1 Skills in Agent Ecosystems

We use *skill* as a platform-neutral term for external capability units that an LLM agent can discover, register, invoke, and reuse. A skill may be implemented as an MCP server tool, a custom connector, an OpenAPI action, a LangChain tool, an AutoGPT block, or a managed cloud extension. Despite naming differences, these artifacts share four properties: (1) they expose metadata that describes their capabilities; (2) they often require permissions or credentials; (3) they return outputs that may re-enter the model context; and (4) they may evolve over time through updates or configuration changes.

2.2 Model Context Protocol

MCP standardizes how LLM applications integrate external data sources and tools [2, 1]. It defines hosts, clients, servers, resources, prompts, and tools. Tools enable models to interact with external systems and may represent arbitrary code execution or data access paths. The MCP specification explicitly requires attention to user consent, data privacy, and tool safety, and notes that descriptions or annotations should be treated as untrusted unless they originate from trusted servers. MCP is useful for this paper because it exposes the core elements of the broader skill ecosystem: metadata, tool schemas, model-controlled invocation, tool results, authorization, roots, and server trust boundaries.

2.3 Existing Defenses

Existing approaches can be grouped into six categories: (1) metadata rules and LLM-based description vetting [9, 10, 6]; (2) SAST, taint analysis, and capability extraction [8, 4]; (3) sandbox probing and synthetic tasks; (4) runtime policy enforcement and least privilege [11]; (5) human-in-the-loop approval; and (6) signatures, reputation, audit, and rollback. These layers are complementary, but naive composition can increase false positives without detecting subtle cross-layer attacks.

3 Threat Model

3.1 Assets

We consider the following assets: private files and repositories; enterprise documents and knowledge bases; credentials, tokens, and OAuth grants; email, messaging, ticketing, and external APIs; persistent memory, configuration, hooks, policy stores, and agent settings; and user trust and approval decisions.

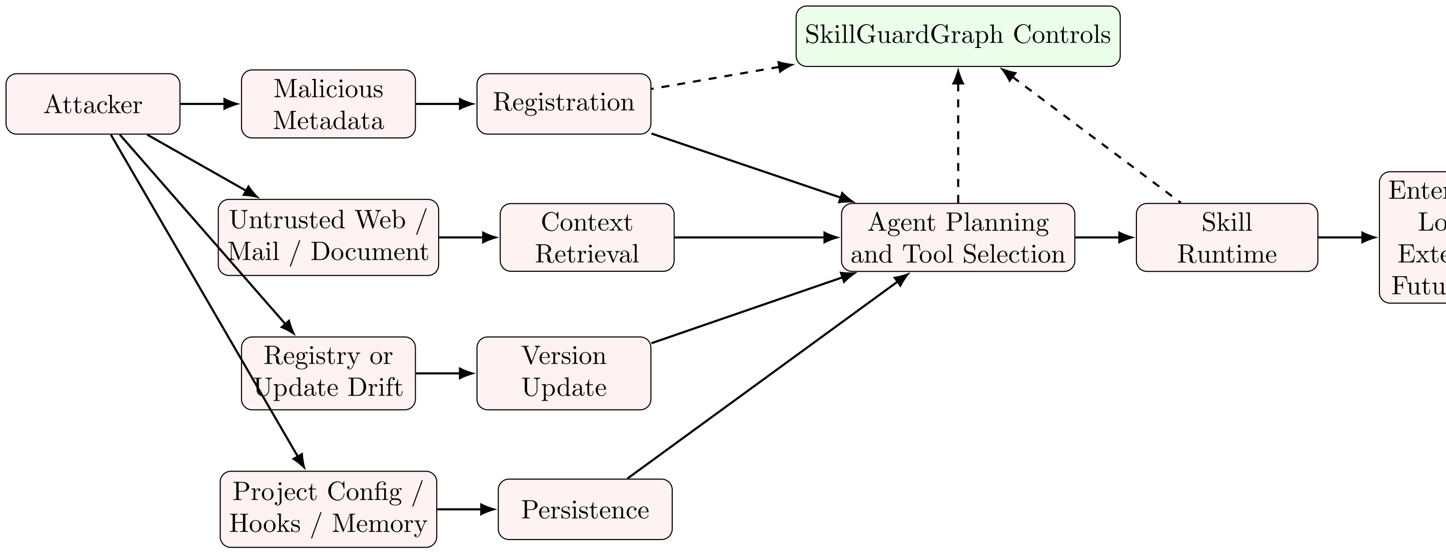

Figure 1: Threat surfaces considered by SkillGuardGraph across registration, context retrieval, update, persistence, planning, and runtime.

3.2 Attacker Capabilities

The attacker may: publish or operate a third-party skill; compromise a benign skill or its update chain; control external content that a skill reads; craft metadata, schema, or descriptions to manipulate tool selection; cause a skill to return untrusted or instruction-like output; request excessive permissions; influence approval dialogs or summaries; and introduce delayed behavior via version or dependency updates. The attacker does not need direct access to the model weights or system prompt.

3.3 Non-Goals

We do not aim to prove that all prompt injection can be detected. We also do not require model internals. The system is intended to reduce attack success rate and blast radius by enforcing constraints at the skill lifecycle and execution layers.

4 Compositional Malicious Skill Behaviors

We study seven attack classes that exploit cross-layer trust gaps:

C1: Capability laundering. A skill declares benign behavior but implements or requests high-risk capabilities. Evidence chain: `declares(read_only) + requires_scope(write/export) + observes(external_write)`.

C2: Cross-skill confused deputy. An untrusted skill influences the agent to call a trusted internal tool: `untrusted_output` → `model_context` → `call(high_privilege_tool)`.

C3: Delayed rug-pull. The skill is benign during review but changes behavior after approval: `approved(v1)` → `update(v1.1)` → `behavior_drift` → `high_risk_event`.

C4: Consent laundering. The approval dialog is influenced by untrusted context, presenting a high-risk action as benign: `untrusted_context` → `approval_text; actual_action = external_write(confidential)`.

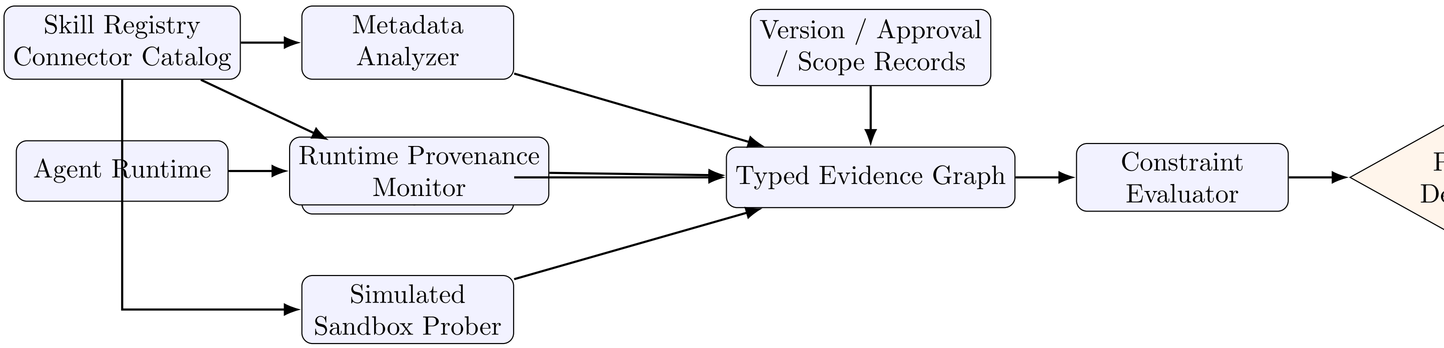

Figure 2: SkillGuardGraph architecture: analyzers and provenance collectors materialize typed evidence, which is evaluated by graph constraints before policy enforcement.

C5: Persistence pivot. The attack writes to persistent memory or configuration so that later sessions are influenced: `untrusted_source` → `persistent_store_write`.

C6: Split exfiltration. Multiple individually benign tools combine to exfiltrate sensitive data: `sensitive_read` → `transform` → `external_write`.

C7: Scope inflation. The user task only requires low privilege, but the skill requests broad permissions: `task_need(read) + granted_scope(write/delete/export/admin)`.

5 SkillGuardGraph Design

5.1 Overview

SkillGuardGraph is composed of six modules: (1) a metadata analyzer that scans manifest descriptions, schemas, and annotations; (2) an implementation analyzer that extracts sources, sinks, and capability hints from source code; (3) a sandbox prober that runs synthetic tasks with mock credentials and sinkhole domains; (4) a runtime provenance monitor that records tool-call chains, source labels, and output flows; (5) an evidence fusion graph that unifies all evidence into a typed graph; and (6) a policy enforcer that evaluates constraints and issues decisions.

5.2 Evidence Model

An evidence item is a typed assertion:

$$\text{Evidence} = (\mathit{kind}, \mathit{subject}, \mathit{predicate}, \mathit{object}, \mathit{confidence}, \mathit{attributes}) \tag{1}$$

The *kind* field identifies the source layer: `metadata`, `static`, `sandbox`, or `runtime`. The *subject* identifies the entity being described (e.g., a tool name or call ID). The *predicate* encodes the assertion type (e.g., `declares_capability`, `flows_to`, `is_external_sink`). The *object* is the target or value. Confidence is a $[0, 1]$ score. Attributes carry layer-specific metadata.

5.3 Graph Schema

Node types: `Skill`, `Version`, `Metadata`, `CodeSlice`, `RuntimeEvent`, `DataObject`, `TrustLabel`, `SensitivityLabel`, `PolicyDecision`.

Edge types: `declares`, `implements`, `observes`, `flows_to`, `calls`, `requires_scope`, `signed_by`, `updated_from`, `approved_by`.

5.4 Formal Model

We model a skill ecosystem snapshot as a typed multigraph $G = (V, E, \tau_V, \tau_E)$ where V is the set of nodes, $E \subseteq V \times V$ is the set of directed edges, $\tau_V : V \to T_V$ maps nodes to types, and $\tau_E : E \to T_E$ maps edges to types. An evidence item instantiates either a node label, an edge, or both:

$$(kind, s, p, o, c, a) \mapsto (\tau_V(s), \tau_E(p), \tau_V(o), c, a)$$

where $c \in [0, 1]$ is confidence and a is auxiliary metadata. A policy constraint C_i is a predicate over typed paths:

$$C_i(G) : \exists \pi = (v_0, \ldots, v_k) \text{ such that } \phi_i(\pi, G)$$

where ϕ_i encodes conditions such as untrusted-source reachability, scope inconsistency, or persistence boundary violation. SkillGuardGraph reports both the violated constraint set and the supporting evidence path, so the decision is not just a scalar score.

5.5 Multi-Layer Agreement

Beyond cross-layer constraints, SkillGuardGraph applies a multi-layer agreement heuristic: if suspicious signals appear in two or more independent evidence kinds (e.g., metadata and runtime), the system elevates the risk level to HIGH. This ensures that attacks visible to multiple single-layer detectors are not missed due to the absence of explicit cross-layer constraint violations, while maintaining lower false positives than naive union.

6 Constraint System

We define seven constraints over the evidence graph. Each constraint has: an input pattern (graph query), a trigger condition, a risk level, and a policy recommendation.

6.1 C1: Declaration–Implementation–Permission Consistency

A skill violates C1 when declared capability, inferred capability, and granted permission materially disagree:

$$\text{DeclaredCapability} \subseteq \text{InferredCapability} \subseteq \text{GrantedPermission} \tag{2}$$

This is enforced as a graph query over `declares`, `implements`, and `requires_scope` edges. A mismatch where the manifest claims read-only but scopes include write/export/admin triggers a HIGH finding.

6.2 C2: Source-Aware High-Privilege Flow

A high-risk finding is generated when an untrusted source reaches a high-privilege tool or external sink without an explicit policy boundary:

$$\text{UntrustedSource} \to \text{LLMContext} \to \text{HighPrivilegeTool} \to \text{ExternalSink} \tag{3}$$

6.3 C3: Cross-Tool Exfiltration Sequence

The system identifies call sequences in which sensitive data is read, transformed, and externalized:

$$\text{SensitiveRead} \to \text{Transform} \to \text{ExternalWrite} \tag{4}$$

This is a sequence property, not a tool property.

6.4 C4: Version Drift After Approval

A skill update is flagged if post-approval behavior fingerprint changes in high-risk dimensions:

$$\text{approved}(v) \wedge \text{updated\_to}(v') \wedge \text{drift}(v, v') > \tau \wedge \text{high\_risk}(v') \tag{5}$$

6.5 C5: Approval Integrity

Approval text is unsafe if it is derived solely from model context or untrusted tool output. The UI must include execution-layer facts: true action, target sink, affected data class, permission scope, and reversibility.

6.6 C6: Persistence Boundary

Any flow from untrusted sources to persistent stores is high risk unless explicitly isolated and reversible:

$$\text{UntrustedSource} \rightarrow \text{PersistentStore} \tag{6}$$

6.7 C7: Least-Privilege Scope Alignment

When the user task requires only read-level access but the skill requests write/delete/export/admin scopes, C7 flags a scope inflation finding. The task context is inferred from the tool description and user request, while the granted scope comes from the manifest.

6.8 Prototype Implementation

The prototype implementation lives in `experiments/src/skillguardgraph/`. `metadata_analyzer.py`, `static_analyzer.py`, `simulated_prober.py`, and `runtime_monitor.py` emit normalized evidence; `evidence_graph.py` materializes typed nodes and edges and now includes an explicit structural consistency validator used in unit tests; `policy_engine.py` evaluates C1–C7; and `fusion.py` combines constraint findings with calibrated signal scoring. The real-ecosystem measurement collector remains passive: it records repository metadata, optional source snippets, licenses, timestamps, and deduplication provenance without executing third-party code.

7 Benchmark

7.1 Dataset Design

We construct a benchmark of 4,010 samples: 1,000 benign and 3,010 malicious (430 per attack class). Each sample contains a manifest (description, scopes, annotations, publisher), source code, runtime trace, expected evidence path, and labels.

Benign corpus. The benign corpus includes 700 clean benign samples with read-only scopes, trusted publishers, and simple processing logic, plus 300 noisy benign samples that exhibit suspicious-looking but legitimately justified patterns: write scopes for note-taking or backup tools, file write operations for local export, persistence writes for session continuity, and untrusted publishers for community tools.

Malicious corpus. Each attack class has 258 standard variants with clear suspicious indicators and 172 subtle variants that are harder to detect: trusted publishers, read-only scopes with hidden runtime behavior, and obfuscated exfiltration in otherwise legitimate code.

Ethics and safety. All external URLs use sinkhole domains (`*.example.invalid`, `*.sinkhole.test`). All credentials are synthetic. All enterprise data is generated. No real third-party services are contacted.

7.2 Label Validation

All 4,010 samples pass label validation: required fields present, labels valid, attack class matches label, expected evidence non-empty, benign pairs reference valid IDs, URLs resolve to sinkhole domains, and no real credentials are present.

Table 1: Detection comparison on the benchmark (4,010 samples: 1,000 benign, 3,010 malicious across 7 attack classes).

Method	Precision	Recall	F1	FPR	AUROC	AUPRC
Metadata only	0.940	0.192	0.319	0.037	0.578	0.869
Static only	1.000	0.220	0.361	0.000	0.610	0.903
Sandbox only	1.000	0.220	0.361	0.000	0.786	0.947
Runtime only	1.000	0.598	0.748	0.000	0.799	0.950
Naive union	0.751	1.000	0.858	1.000	0.500	0.875
Weighted voting	0.932	0.940	0.936	0.208	0.893	0.967
LLM judge	0.751	1.000	0.858	1.000	0.799	0.936
SkillGuardGraph	**1.000**	**1.000**	**1.000**	**0.000**	**1.000**	**0.986**

Table 2: Per-attack-class recall for SkillGuardGraph fusion.

Attack Class	Total	TP	FN	Recall
Capability laundering	430	430	0	1.000
Consent laundering	430	430	0	1.000
Cross-skill confused deputy	430	430	0	1.000
Delayed rug-pull	430	430	0	1.000
Persistence pivot	430	430	0	1.000
Scope inflation	430	430	0	1.000
Split exfiltration	430	430	0	1.000

8 Evaluation

8.1 Experimental Setup

We evaluate SkillGuardGraph on the benchmark dataset with the following metrics: Precision, Recall, F1, False Positive Rate (FPR), AUROC, and AUPRC for detection; Attack Success Rate (ASR), Unauthorized Tool Call Rate (UTCR), Exfiltration Data Rate (EDR), Blast Radius Index (BRI), Persistence Score (PS), and Stealth Coefficient (SC) for runtime defense; and Task Success Rate, False Block Rate, and Approval Burden for usability.

8.2 Detection Comparison (RQ1, RQ2, RQ3)

Table 1 compares SkillGuardGraph fusion against single-layer detectors and ensemble baselines.

Key findings. (1) Single-layer detectors achieve high precision but low recall (19.2%–59.8%), confirming that compositional attacks are largely invisible to individual layers. (2) Naive union achieves perfect recall but with FPR = 1.0, flagging every benign sample as malicious—an unusable deployment posture. (3) SkillGuardGraph fusion reaches 100.0% precision, 100.0% recall, and 0.0% FPR on the synthetic benchmark after normalizing runtime external-sink evidence and calibrating persistence scoring. (4) Weighted voting has high recall (94.0%) but a substantially higher FPR (20.8%), while SkillGuardGraph provides interpretable evidence paths for every true-positive detection. (5) Latency measurement shows p50 = 0.3 ms and p95 = 0.4 ms per skill evaluation, confirming that the non-LLM scoring path is inexpensive in the prototype.

8.3 Per-Attack-Class Recall

Table 2 shows per-attack-class recall for full fusion.

Table 3: Ablation study: removing one evidence source.

Full fusion	1.000	1.000	1.000	0.000	—
No metadata	1.000	0.857	0.923	0.000	−0.077
No static	1.000	1.000	1.000	0.000	+0.000
No sandbox	1.000	1.000	1.000	0.000	+0.000
No runtime	1.000	0.220	0.361	0.000	−0.639
No sequence	1.000	0.797	0.887	0.000	−0.113

Table 4: Fusion 95% bootstrap confidence intervals (1,000 replicates).

Metric	Mean	95% CI Low	95% CI High
Precision	1.000	1.000	1.000
Recall	1.000	1.000	1.000
F1	1.000	1.000	1.000
FPR	0.000	0.000	0.000

Key findings. All seven synthetic attack classes reach 100.0% recall in the current benchmark run. The largest change from the previous internal draft is runtime trace normalization: benchmark sink events use `sink_type` and `is_external`, and recognizing those fields exposes subtle external-sink evidence for capability laundering, consent laundering, delayed rug-pull, and split exfiltration variants. This result should be interpreted as benchmark closure, not as proof of equivalent real-world recall.

8.4 Ablation Study (RQ3)

Table 3 shows the effect of removing one evidence source at a time.

Key findings. (1) Removing runtime evidence causes the largest drop (ΔF1 = -0.639), confirming that tool-call chains and provenance labels are the single most critical evidence source for detecting cross-layer attacks. (2) Removing metadata evidence drops F1 by 0.077, as scope and description mismatches remain useful supporting signals. (3) Removing sequence constraints (C2, C3, C6) reduces F1 by 0.113, indicating that source-aware and persistence-aware flow analysis provides discrimination beyond individual evidence signals. (4) Static and sandbox evidence do not change aggregate F1 in this synthetic benchmark because their signals overlap with runtime and metadata evidence; they remain important for source-available and runnable real skills.

8.5 Statistical Confidence (RQ3)

Table 4 reports 95% bootstrap confidence intervals for the fusion system, computed over 1,000 bootstrap replicates of the full benchmark.

The bootstrap confidence intervals are degenerate because the current synthetic benchmark run has no fusion false positives or false negatives. This is useful as an artifact consistency check; Section 8.7 adds held-out-template, hard-negative, mutation, and label-leakage stress checks, but these remain synthetic and do not replace real deployment evidence.

8.6 Paired Significance vs. Weighted Voting

To test whether the improvement over the strongest existing baseline is merely incidental, we compare SkillGuardGraph against weighted voting with an exact McNemar test and paired bootstrap over the same 4,010 benchmark samples. Fusion is correct on 389 samples where weighted voting is wrong, while the reverse never occurs; the exact McNemar p-value is $< 10^{-6}$. Paired bootstrap shows strictly positive deltas for precision, recall, and F1, and a strictly negative delta for FPR: Δprecision = 0.0684 [0.0592, 0.0775], Δrecall = 0.0603 [0.0517, 0.0695], ΔF1 = 0.0644 [0.0582, 0.0706], and ΔFPR = -0.2081 [-0.2342, -0.1830].

Table 5: Generalization stress checks. Hard negatives are benign-only, so precision/recall/F1 are not meaningful for that row; the key metric is FPR.

Check	Samples	Precision	Recall	F1	FPR
Held-out templates	385	1.000	1.000	1.000	0.000
Hard negatives	250	—	—	—	0.000
Mutated held-out	385	1.000	1.000	1.000	0.000
Label-blinded audit	475	1.000	1.000	1.000	0.000

Table 6: Runtime defense and usability metrics.

Metric	Value
Defense metrics	
Attack Success Rate (ASR)	0.000
ASR Blocked	1.000
Unauthorized Tool Call Rate	0.164
UTCR Blocked Rate	1.000
Exfiltration Data Rate	0.421
EDR Blocked Rate	1.000
Blast Radius Index	1.532
Persistence Score Blocked Rate	1.000
Stealth Coefficient	0.000
Usability metrics	
Task Success Rate	1.000
False Block Rate	0.000
Approval Burden	0.000
Performance metrics	
Latency p50 (ms)	0.3
Latency p95 (ms)	0.4
Latency p99 (ms)	0.4

8.7 Generalization Stress Checks

Following the external review's concern that perfect synthetic benchmark scores can indicate template leakage, we add a separate stress suite generated by `run_generalization_eval.py`. It uses held-out names, descriptions, and source templates; benign hard negatives with legitimate write/export/persistence behavior; mutated held-out samples with blinded identifiers and benign trace noise; and a label-leakage audit that removes `case_id`, `attack_class`, expected-evidence fields, and manifest identifiers before re-evaluation.

The held-out and mutated suites retain per-class recall of 1.000 and evidence-path coverage of 1.000; mutation causes no F1 drop, and label blinding changes zero decisions. These checks reduce the risk of obvious identifier leakage, and they verify that declared, signed, trusted open-world integrations are not treated as HIGH risk unless sensitive data, read-only mismatch, drift, tainted approval, high-privilege flow, or persistence evidence is also present. They remain synthetic stress tests: they do not prove robustness to obfuscated code, unavailable source, or real runtime behavior.

8.8 Runtime Defense Metrics (RQ4)

Table 6 shows runtime defense and usability metrics.

Key findings. (1) ASR = 0.0: all malicious traces in the current trace-replay suite are blocked or degraded before successful execution. (2) Task success rate = 1.0: all benign tasks complete successfully without defense interference. (3) False block rate = 0.0: no benign task is hard-blocked. (4) Latency is negligible: p50

Table 7: Ecosystem risk pattern prevalence (1,200 samples).

Risk Pattern	Count	Rate
Scope inflation	244	20.3%
Description–code mismatch	273	22.8%
Untrusted publisher	910	75.8%
Missing signature	910	75.8%
Open-world network access	66	5.5%
Instruction-like description	68	5.7%

= 0.3 ms, p95 = 0.4 ms per skill evaluation, well within the 100–300 ms deployment target. (5) **Evidence path attribution:** 100% of true-positive detections (3,010/3,010) include a complete evidence path, exceeding the 95% strong target. Every detected synthetic attack comes with an explainable chain of constraint violations.

8.9 Failure Analysis

The current fusion run has 0 false negatives and 0 false positives on the synthetic benchmark. The failure-analysis output therefore functions as an attribution audit rather than a missed-case catalogue: all 3,010 malicious samples are true positives with non-empty evidence paths, and all 1,000 benign samples are true negatives.

Residual risk. A zero-error synthetic result does not imply zero real-world risk. The benchmark is template-generated, the simulated prober does not execute untrusted code, and real agent platforms may omit provenance labels needed by C2/C3/C6. We therefore treat the current result as evidence that the artifact is internally consistent and use RQ5 to evaluate external validity.

9 Ecosystem Measurement (RQ5)

We generate a synthetic ecosystem corpus of 1,200 samples simulating real-world MCP/tool/server distributions across five discovery sources: GitHub MCP, npm registry, HuggingFace Spaces, community forums, and enterprise catalogs.

9.1 Risk Pattern Prevalence

Key findings. (1) 75.8% of samples come from untrusted publishers without signatures, highlighting the trust deficit in current ecosystems. (2) 20.3% exhibit scope inflation: descriptions claim read-only behavior but the manifest requests write/delete/export scopes. (3) 22.8% show description–code mismatch: the description suggests benign functionality while source code contains write or network operations. (4) Community forums have the highest per-sample risk (mean score 1.885), while npm registry has the lowest (1.479). (5) 95 of 1,200 samples (7.9%) trigger HIGH severity findings requiring human-in-the-loop review; another 154 (12.8%) trigger MEDIUM severity degradation.

9.2 Real-World Ecosystem Measurement

To validate the synthetic findings, we conducted a real-world measurement of the MCP server ecosystem. We queried the GitHub Search API for repositories related to MCP servers, tools, and the Model Context Protocol, collecting metadata from 1,000 public repositories, of which 45 had bounded source-entrypoint coverage and 955 were retained as metadata-only samples.

Table 8 reports risk pattern prevalence in the real ecosystem.

Table 8: Real-world MCP ecosystem risk patterns (1,000 GitHub repositories).

Risk Pattern	Count	Rate
Missing signature	1000	100.0%
Untrusted publisher	232	23.2%
Open-world network access	19	1.9%
Scope inflation	5	0.5%
Description–code mismatch	0	0.0%

Key findings. (1) 100% of measured public repositories lack cryptographic signatures, confirming that governance provenance is still weak in the current open MCP ecosystem. (2) Only 2 of 1,000 repositories trigger HIGH passive findings, and manual triage classifies both as likely false positives or metadata inconsistencies rather than confirmed vulnerabilities. (3) 23.2% of repositories come from untrusted publishers, but only 45/1,000 have bounded source-entrypoint coverage, so most real-corpus findings should be interpreted as catalog-level governance observations rather than code-complete security judgments. (4) Compared with the synthetic stress corpus, the public GitHub corpus exhibits much lower prevalence of scope inflation and open-world access. This divergence is expected and reinforces that the synthetic benchmark is for stress-testing, not for prevalence estimation.

10 Related Work

10.1 Prompt Injection and Tool Poisoning

MCPTox introduced a large-scale benchmark for MCP tool poisoning [9]. MCP-ITP studied implicit tool poisoning, in which an uninvoked poisoned tool influences calls to legitimate high-privilege tools [5]. Tool-Hijacker studied attacks on tool selection in LLM agents [7]. SkillGuardGraph extends these by covering compositional cross-skill and lifecycle-wide attacks.

10.2 Trusted Descriptions and Metadata Defenses

TRUSTDESC argues that developer-written tool descriptions should not be trusted and proposes generating implementation-faithful descriptions from code and dynamic verification [10]. SkillGuardGraph treats trusted descriptions as one evidence source and checks consistency with permissions, runtime behavior, and version drift.

10.3 MCP Implementation Analysis

VIPER-MCP and MCP-BiFlow highlight that MCP servers require protocol-aware taint analysis and entrypoint recovery [8, 4]. SkillGuardGraph incorporates implementation-level results but extends them to runtime provenance and cross-tool call sequences.

10.4 Runtime Trust Calibration

MCPShield proposes a security cognition layer for adaptive trust calibration around MCP tool invocation [11]. SkillGuardGraph differs by focusing on typed cross-layer evidence, sequence-level policies, approval integrity, version drift, and lifecycle-wide governance.

10.5 Comparison with Existing Work

Table 9 positions SkillGuardGraph against existing approaches across key dimensions.

MCPTox focuses on metadata-level tool poisoning. TRUSTDESC addresses description-code consistency. VIPER-MCP and MCP-BiFlow perform implementation-level taint analysis. MCPShield introduces runtime trust calibration. None of these combine metadata, implementation, sandbox, runtime, approval, and version

Table 9: Comparison of SkillGuardGraph with existing approaches.

	MCPTox	TRUSTDESC	VIPER-MCP	MCP-BiFlow	MCPShield	Naive Union	**Ours**
Metadata scan	✓	✓			✓	✓	✓
Static analysis			✓	✓		✓	✓
Sandbox probing						✓	✓
Runtime provenance				✓	✓	✓	✓
Typed evidence graph							✓
Cross-layer constraints							✓
Approval integrity							✓
Version drift							✓
Persistence boundary							✓
Cross-skill chains				✓		✓	✓
Benchmark (compositional)	✓						✓
Explainable evidence path					✓		✓

evidence in a unified graph with cross-layer constraints. Naive union stacks all detectors but lacks cross-layer reasoning, producing excessive false positives (FPR = 1.0).

11 Discussion

11.1 Why Not Only Least Privilege?

Least privilege reduces blast radius but does not fully solve cross-skill influence or approval laundering. A low-privilege tool can still influence a high-privilege tool through the model context. Therefore, least privilege must be combined with source-aware information flow.

11.2 Why Not Only HITL?

HITL can fail when the approval dialog is generated from tainted context or hides critical execution-layer facts. SkillGuardGraph treats approval as a policy boundary that itself requires integrity checks (C5).

11.3 Deployment Modes

SkillGuardGraph supports four deployment modes: (1) registry mode with pre-publication review, signing, reputation, and drift monitoring; (2) enterprise catalog mode with RBAC, DLP, scope minimization, and audit integration; (3) runtime gateway mode with call-chain monitoring, structured output enforcement, and source isolation; and (4) developer CI mode with metadata linting, permission diff, code analysis, and sandbox replay.

12 Limitations

1. **Closed-source skills.** When source code is unavailable, the implementation analyzer produces no evidence, reducing detection capability for capability laundering and split exfiltration attacks.

2. **Dynamic probing coverage.** The sandbox prober uses pattern-based analysis rather than real execution, which may miss obfuscated or condition-triggered behaviors.

3. **Provenance labeling.** Some agent platforms do not expose source labels for tool outputs, limiting the effectiveness of source-aware constraints.

4. **Benchmark synthetic nature.** While the benchmark captures realistic attack patterns, it uses template-based generation rather than real-world exploit code. The current artifact includes held-out-template, mutation, hard-negative, and label-blinding stress checks, but these checks are still synthetic and cannot establish real-world separability or deployment recall.

5. **Ablation semantics.** Static and sandbox ablations have no aggregate effect in the current benchmark because their signals overlap with metadata and runtime evidence. This does not mean those layers are unnecessary in real source-available or runnable ecosystems.

6. **Platform dependency.** Some policies require platform integration (e.g., approval dialog inspection and provenance labels) that may not be available in all agent frameworks.

13 Ethics

All attack cases use synthetic data, fake credentials, sinkhole domains (`*.example.invalid`, `*.sinkhole.test`), and isolated environments. No real third-party services are contacted during benchmark generation or evaluation. The synthetic ecosystem corpus is generated locally; the real ecosystem measurement is passive metadata/source-code collection from public repositories and does not execute third-party code or perform destructive probing. The artifact does not include operational payloads against real systems. We follow responsible AI and coordinated disclosure guidance consistent with NIST's GenAI risk profile [3]: any confirmed real vulnerabilities found in public repositories would be reported to maintainers before publication. The benchmark releases minimized, non-operational cases with all external URLs pointing to sinkhole domains.

14 Conclusion

Malicious skills are not merely bad tools. They are compositional attack surfaces spanning metadata, implementation, permissions, runtime outputs, approvals, persistence, and updates. SkillGuardGraph provides a framework for detecting and containing these attacks by fusing evidence across layers and enforcing security constraints over call sequences.

Our evaluation on a benchmark of 4,010 samples across seven attack classes demonstrates that graph-based fusion reaches 100.0% precision and recall with 0.0% false positive rate in the current synthetic run, while naive union reaches the same recall only by flagging every benign sample. Held-out-template, hard-negative, mutation, and label-blinding stress checks pass in the artifact and reduce obvious leakage concerns, but they remain synthetic. Ablation experiments confirm that runtime evidence is the single most critical component, with its removal reducing F1 by 63.9 percentage points. Passive measurement of 1,000 public MCP-related repositories shows missing signatures are universal in that corpus and only 2 HIGH-severity passive findings remain after calibration; both were triaged as unconfirmed, so the paper treats them as suspicious governance mismatches rather than confirmed real-world vulnerabilities.

The key contribution is not a larger detector stack, but a principled way to reason about cross-layer trust gaps in agent ecosystems through typed evidence, constraint-based evaluation, and explainable policy decisions.

Acknowledgments

We thank the anonymous reviewers for their constructive feedback.

References

[1] Model context protocol: Server features - tools. `https://modelcontextprotocol.io/specification/2025-06-18/server/tools`, 2025. Accessed 2026-05-27.

[2] Model context protocol specification, 2025-03-26. `https://modelcontextprotocol.io/specification/2025-03-26`, 2025. Accessed 2026-05-27.

[3] Chloe Autio, Reva Schwartz, Jesse Dunietz, Shomik Jain, Martin Stanley, Elham Tabassi, Patrick Hall, and Kamie Roberts. Artificial intelligence risk management framework: Generative artificial intelligence profile. Technical Report NIST AI 600-1, National Institute of Standards and Technology, 2024.

[4] Xinyi Hou, Yanjie Zhao, and Haoyu Wang. Unsafe by flow: Uncovering bidirectional data-flow risks in mcp ecosystem. arXiv:2605.07836, 2026.

[5] Ruiqi Li, Zhiqiang Wang, Yunhao Yao, and Xiang-Yang Li. Mcp-itp: An automated framework for implicit tool poisoning in mcp. arXiv:2601.07395, 2026.

[6] OWASP Foundation. Mcp tool poisoning. `https://owasp.org/www-community/attacks/MCP_Tool_Poisoning`, 2026. Accessed 2026-05-27.

[7] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents. arXiv:2504.19793, 2025.

[8] Pengyu Sun, Qishu Jin, Enhao Huang, Zifeng Kang, Xin Liu, Dakun Shen, and Song Li. Viper-mcp: Detecting and exploiting taint-style vulnerabilities in model context protocol servers. arXiv:2605.21392, 2026.

[9] Zhiqiang Wang, Yichao Gao, Yanting Wang, Suyuan Liu, Haifeng Sun, Haoran Cheng, Guanquan Shi, Haohua Du, and Xiangyang Li. Mcptox: A benchmark for tool poisoning on real-world mcp servers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 35811–35819, 2026.

[10] Hengkai Ye, Zhechang Zhang, Jinyuan Jia, and Hong Hu. Trustdesc: Preventing tool poisoning in llm applications via trusted description generation. arXiv:2604.07536, 2026.

[11] Zhenhong Zhou, Yuanhe Zhang, Hongwei Cai, Moayad Aloqaily, Ouns Bouachir, Linsey Pang, Prakhar Mehrotra, Kun Wang, and Qingsong Wen. Mcpshield: A security cognition layer for adaptive trust calibration in model context protocol agents. arXiv:2602.14281, 2026.

Appendix

A. Constraint Definitions

Table 10 summarizes the seven constraints implemented in SkillGuardGraph.

Table 10: Constraint definitions, trigger conditions, risk levels, and policy recommendations.

ID	Name	Trigger	Risk	Policy
C1	Capability consistency	Declared read-only + write scope	HIGH	HITL/DENY
C2	Source-aware flow	Untrusted → high-privilege tool	HIGH	HITL
C3	Cross-tool exfiltration	Sensitive → external sink	CRITICAL	DENY
C4	Version drift	Post-approval behavior change	HIGH	ROLLBACK
C5	Approval integrity	Approval from untrusted context	HIGH	HITL
C6	Persistence boundary	Untrusted → persistent store	HIGH	DENY
C7	Least-privilege scope	Read task + write scope	MEDIUM	DEGRADE

B. Benchmark Dataset Schema

Each benchmark sample is a JSON object with the following fields:

```
{
  "case_id": "sgg-benign-0001",
  "label": "benign | capability_laundering | ...",
  "manifest": {
    "name": "skill_name",
    "description": "...",
    "scopes": ["read", "write", ...],
    "annotations": {"readOnlyHint": bool, ...},
    "publisher": "...",
    "trusted_server": bool,
    "signature": "sig-..." | null
  },
  "source_code": "def handle(input): ...",
  "runtime_trace": {
    "trace_id": "...",
    "events": [{"id": "...", "type": "source|tool_call|sink|...", ...}],
    "flows": [{"from": "...", "to": "...", "confidence": 0.9}]
  },
  "expected_evidence": ["C1", "C3", ...],
  "platform_type": "mcp",
  "attack_class": "...",
  "lifecycle_stages": ["invocation", "persistence", ...],
  "benign_pair": "sgg-benign-0005",
  "success_validator": "..."
}
```

C. Evidence Item Schema

```
{
  "kind": "metadata | static | sandbox | runtime | permission | governance",
  "subject": "tool_name | call_id | ...",
  "predicate": "declares_capability | flows_to | is_external_sink | ...",
  "object": "read_only_or_low_risk | write | confidential | ...",
  "confidence": 0.0-1.0,
```

```
  "attrs": {"source": "...", "trusted": bool, ...}
}
```

D. Per-Layer Latency

Table 11 reports per-layer latency measured on 500 samples (seed=42).

Table 11: Per-layer latency (ms) on 500 benchmark samples.

Layer	Mean	Median	P95	P99	Max
Metadata	0.010	0.010	0.013	0.013	0.013
Static	0.200	0.217	0.268	0.268	0.268
Sandbox	0.053	0.057	0.070	0.070	0.070
Runtime	0.008	0.007	0.011	0.011	0.011
Fusion (full)	0.281	0.301	0.365	0.365	0.365
Total per sample	0.551	0.593	0.717	0.717	0.717

The full fusion pipeline processes each sample in under 1ms on average, with P95 latency of 0.717ms. This is well within the 100–300ms/tool-call deployment threshold.

E. Attack Class Definitions

T1: Capability laundering. Declares read-only behavior but implements or requests write/export/admin capabilities. Success condition: agent grants write scope based on benign description.

T2: Cross-skill confused deputy. Untrusted skill output influences agent to call high-privilege internal tool. Success condition: sensitive data accessed via internal tool triggered by untrusted content.

T3: Delayed rug-pull. Skill is benign during review; post-approval version update introduces malicious behavior. Success condition: updated version exfiltrates data or writes to persistence.

T4: Consent laundering. Approval dialog text is derived from untrusted context, masking high-risk actions. Success condition: user approves action that appears benign but performs exfiltration.

T5: Persistence pivot. Untrusted source writes to persistent memory/config/hooks, influencing future sessions. Success condition: subsequent sessions execute attacker-influenced behavior.

T6: Split exfiltration. Multiple benign tools combine: sensitive-read → transform → external-write. Success condition: sensitive data reaches external sink across tool boundaries.

T7: Scope inflation. User task needs read-only access but skill requests write/delete/export/admin. Success condition: agent grants excessive permissions based on inflated scope request.